

PhD Project Proposal

EirVah: A Scalable, Cost-Efficient and Open Reference Architecture for Unified Namespace in Industrial IoT

William Francis Stack

Supervisor: Aleš Janota

University of Žilina / Dept. of Control and Information Systems, FEIT,
Univerzitná 8214/1, 010 26 Žilina, Slovak Republic

May 10, 2026

Abstract

The digital transformation of industrial systems has intensified the need for architectures capable of integrating heterogeneous devices, protocols, and applications into scalable data ecosystems. Traditional hierarchical models provide structure but often lead to fragmented data flows, limited interoperability, and increased system complexity.

Unified Namespace (UNS) has emerged as a promising paradigm for real-time, data-centric integration in Industrial IoT. However, there is currently no widely accepted formal reference architecture for UNS, and limited evidence exists that such architectures can be implemented cost-effectively using open-source technologies across edge, cloud, and hybrid environments.

This project proposes *EirVah*, a scalable, cost-efficient, and open reference architecture for Unified Namespace in Industrial IoT. The framework integrates telemetry and event-driven communication within a unified semantic namespace and is designed for consistent operation across edge, private cloud, public cloud, and hybrid infrastructures.

The architecture will be implemented using open source technologies and evaluated through a near-real cyber-physical system testbed with emulated shop-floor communication. The research aims to demonstrate that UNS can be formalized as a reproducible, observable, and deployment-agnostic architecture suitable for modern industrial systems.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Research Objectives	2
1.5 Research Questions	2
1.6 Proposed Approach	3
1.7 Expected Contributions	3
1.8 Structure of the Proposal	4
2 State of the Art	5
2.1 Introduction	5
2.2 Industrial Integration Architectures and ISA-95	5
2.2.1 Evolution of Industrial Integration Architectures	5
2.2.2 The ISA-95 Model	6
2.2.3 Limitations of Hierarchical Architectures	6
2.2.4 Transition Toward Data-Centric Architectures	7
2.3 Industrial Communication and Protocol Diversity	7
2.4 MQTT and AMQP in Industrial Systems	8
2.4.1 MQTT and Telemetry Communication	8
2.4.2 AMQP and Event-Driven Communication	8
2.4.3 MQTT and AMQP as Complementary Paradigms	9
2.4.4 Message Semantics and Data Consistency	9
2.5 Implications for Industrial Integration	10
2.6 Unified Namespace	10
2.6.1 Concept and Architectural Principles	10
2.6.2 Namespace Structure and Semantic Representation	11

2.6.3	UNS in Industrial Practice	12
2.6.4	Data Flow and Communication Characteristics in UNS	12
2.6.5	Comparison with Alternative Integration Approaches	13
2.6.6	Scalability and Distribution in UNS Architectures	13
2.6.7	Limitations of Current UNS Approaches	14
2.6.8	Emerging Directions and Research Opportunities	14
2.7	Implications for the Proposed Research	15
2.8	Cyber-Physical Systems and Industrial Integration	15
2.9	Edge Computing and Semantic Contextualization	16
2.10	Cloud-Native and Scalable Industrial Architectures	16
2.11	Cost-Efficient and Open Architectures	17
2.12	Research Gap	18
3	Theoretical Framework	20
3.1	Introduction	20
3.2	Cybernetics and System Theory	20
3.3	Cyber-Physical Systems	21
3.4	Observability and System State Representation	21
3.5	Feedback Loops and Control in Distributed Systems	22
3.6	Data-Centric Architectures and System Integration	22
3.7	Towards a Formal Reference Architecture	23
4	EirVah Architecture: A Scalable and Open Reference Model for Unified Namespace	24
4.1	Introduction	24
4.2	Abstract Architecture Foundation	24
4.3	Architectural Overview	26
4.4	Architectural Layers	28
4.4.1	Edge Integration Layer	28
4.4.2	Unified Namespace and Messaging Layer	30
4.4.3	Data and Persistence Layer	30
4.4.4	Decision and Analytics Layer	31
4.4.5	Observability Layer	32
4.4.6	Orchestration and Deployment Layer	32
4.5	Data Flow and System Interaction	33
4.6	Deployment Model	34
4.7	Key Architectural Properties	34
4.8	Summary	35

5	Research Methodology	36
5.1	Introduction	36
5.2	Research Approach	36
5.3	System Implementation Strategy	37
5.4	Infrastructure as Code and Reproducibility	37
5.5	Experimental Design	38
5.6	Observability and Metrics Collection	38
5.7	Validation Strategy	39
5.8	Limitations of the Methodology	39
5.9	Summary	39
6	Evaluation Plan and Expected Contributions	40
6.1	Introduction	40
6.2	Evaluation Objectives	40
6.3	Experimental Scenarios	41
6.3.1	Baseline Scenario	41
6.3.2	Scalability Scenario	41
6.3.3	Failure and Recovery Scenario	41
6.4	Deployment Model Evaluation	41
6.5	Evaluation Metrics	42
6.6	Cost Evaluation and Open-Source Feasibility	43
6.6.1	Cost Model	43
6.6.2	Experimental Cost Measurement	43
6.6.3	Open-Source Stack Evaluation	44
6.6.4	Cost-Performance Trade-offs	44
6.6.5	Expected Outcome	44
6.7	Reproducibility and Infrastructure as Code	44
6.8	Comparative Evaluation	45
6.9	Expected Results	45
6.10	Scientific Contributions	45
6.10.1	Formalization of Unified Namespace	46
6.10.2	Integration of Communication Paradigms	46
6.10.3	Observability-Driven Architecture	46
6.10.4	Deployment-Agnostic and Reproducible Architecture	46
6.10.5	Cost-Efficient Open Architecture	46
6.11	Practical Contributions	46
6.12	Summary	46

7	Dissertation Theses and Research Objectives	48
7.1	Introduction	48
7.2	Dissertation Thesis Statements	48
7.3	Theoretical Objectives	49
7.4	Practical Objectives	49
7.5	Current Research Outputs	50
7.6	Summary	51
8	Conclusion	52

Chapter 1

Introduction

1.1 Background and Context

Industrial systems are undergoing a fundamental transformation driven by the convergence of Industrial Internet of Things (IIoT), cyber-physical systems (CPS), and cloud-native computing. Modern manufacturing environments increasingly rely on continuous data acquisition, real-time processing, and distributed analytics to improve operational efficiency, reliability, and adaptability.

In these environments, large volumes of data are generated by sensors, machines, and enterprise systems across multiple layers. This data must be integrated and made accessible for monitoring, optimization, and decision-making. However, many industrial architectures remain fragmented and difficult to scale.

Traditional approaches are typically based on hierarchical models such as ISA-95, which structure systems into layers but often introduce rigidity and limit real-time data accessibility [1].

To address these challenges, the concept of Unified Namespace (UNS) has emerged as a data-centric integration paradigm. UNS enables systems to interact through a shared, real-time representation of system state, supporting decoupled and scalable communication [2].

1.2 Motivation

Several challenges motivate this research. First, there is no widely accepted formal reference architecture for Unified Namespace systems. Second, current implementations are largely focused on telemetry and do not fully integrate event-driven communication. Third, many industrial solutions rely on proprietary platforms, leading to high costs and vendor dependency.

In addition, modern systems must operate across diverse environments, including edge, private cloud, and public cloud deployments. There is therefore a need for scalable and deployment-agnostic architectures that can operate consistently across these environments.

1.3 Problem Statement

Current Unified Namespace architectures lack formalization, scalability, and integration of multiple communication paradigms. They also provide limited support for cost-efficient deployment and reproducible evaluation.

These limitations hinder their adoption in modern industrial systems requiring real-time integration, flexibility, and scalability.

1.4 Research Objectives

1. To define a formal reference architecture for Unified Namespace in industrial environments.
2. To integrate telemetry-based and event-driven communication within a unified architectural model based on MQTT and AMQP.
3. To design a scalable deployment model using cloud-native technologies, including Kubernetes and k3s.
4. To enable consistent operation of the architecture across edge, private cloud, public cloud, and hybrid environments.
5. To demonstrate the feasibility of cost-efficient implementation using open-source technologies.
6. To ensure reproducible deployment, experimentation, and evaluation through Infrastructure as Code and observability-driven measurement.

1.5 Research Questions

This research is guided by the following questions:

1. How can a Unified Namespace architecture be formally defined as a scalable and deployment-agnostic framework for industrial systems?

2. How can telemetry-based communication and event-driven messaging be integrated within a unified architectural model?
3. How can such an architecture maintain consistent operation across edge, cloud, and hybrid deployment environments?
4. To what extent can open-source technologies enable cost-efficient implementation without compromising performance and scalability?
5. How can observability and Infrastructure as Code be incorporated to support system evaluation, reproducibility, and optimization?

1.6 Proposed Approach

This research proposes *EirVah*, a scalable and open reference architecture for Unified Namespace. MQTT is used for telemetry and system state propagation, while AMQP (RabbitMQ) is used for event-driven communication. Edge systems translate industrial protocols into a unified namespace representation.

The architecture is cloud-agnostic and supports deployment across edge, private cloud, and public cloud environments using Kubernetes-based infrastructure.

1.7 Expected Contributions

The expected contributions of this research are as follows:

1. A formal reference architecture for Unified Namespace in industrial IoT.
2. A unified multi-protocol integration model combining telemetry and event-driven communication.
3. A scalable and deployment-agnostic architectural framework for operation across edge, cloud, and hybrid environments.
4. An observability-driven approach to architectural evaluation, enabling systematic measurement, monitoring, and optimization.
5. A reproducible implementation and evaluation framework based on Infrastructure as Code.
6. A cost-efficient open-source implementation strategy for advanced industrial data integration.

1.8 Structure of the Proposal

The remainder of this proposal is organized as follows. Chapter 2 reviews the state of the art in industrial integration architectures, communication protocols, Unified Namespace, cyber-physical systems, cloud-native deployment, and cost-efficient open architectures. It identifies the key limitations in existing approaches and establishes the research gap addressed by this project.

Chapter 3 presents the theoretical framework of the research, grounding the proposed work in cybernetics, system theory, cyber-physical systems, observability, and data-centric integration. This chapter provides the conceptual basis for understanding the proposed architecture as more than a technical implementation.

Chapter 4 introduces the EirVah architecture as a scalable, open, and deployment-agnostic reference model for Unified Namespace in industrial IoT. It defines the main architectural layers, communication mechanisms, deployment model, and observability capabilities of the framework.

Chapter 5 describes the research methodology used to design, implement, and validate the proposed architecture. It explains the design science approach, implementation strategy, role of simulation, reproducibility through Infrastructure as Code, and the use of observability for metrics collection.

Chapter 6 presents the evaluation plan and expected contributions of the research. It defines the experimental scenarios, deployment model evaluation, measurement framework, cost analysis, reproducibility strategy, and the anticipated scientific and practical contributions of the project.

Chapter 2

State of the Art

2.1 Introduction

The integration of industrial systems has become a central challenge in the context of Industry 4.0, where increasing connectivity, data availability, and system complexity demand new architectural approaches. Modern industrial environments are characterized by heterogeneous systems, diverse communication protocols, and distributed data sources, which complicate interoperability and limit real-time decision-making capabilities [3, 4].

Recent advances in Industrial Internet of Things (IIoT) and cyber-physical systems (CPS) have enabled new forms of data-driven operation, where information flows continuously across different layers of industrial systems. These developments have increased the importance of scalable and interoperable architectures capable of supporting large-scale data integration and real-time coordination [5, 6].

Despite these advances, effective integration remains difficult due to architectural rigidity, data fragmentation, and the coexistence of multiple communication paradigms. Traditional approaches based on hierarchical models such as ISA-95 are increasingly considered insufficient for modern industrial requirements [7, 8].

This chapter reviews the state of the art in industrial integration architectures, communication protocols, and Unified Namespace, with the goal of identifying key limitations and motivating the need for a scalable and cost-efficient architecture.

2.2 Industrial Integration Architectures and ISA-95

2.2.1 Evolution of Industrial Integration Architectures

Industrial integration architectures have evolved from isolated automation systems toward interconnected cyber-physical environments. Early industrial systems were typi-

cally implemented as vertically integrated solutions, where control systems, supervisory platforms, and enterprise applications operated independently or were connected through proprietary interfaces [3].

These architectures were primarily designed for reliability and local control rather than interoperability and scalability. As industrial systems became more complex and interconnected, their limitations became increasingly evident, particularly in relation to data integration and cross-system communication [4].

The introduction of standardized integration models, particularly ISA-95, marked a significant step toward addressing these challenges by providing a structured framework for integrating enterprise and control systems [1].

2.2.2 The ISA-95 Model

The ISA-95 model defines a hierarchical architecture that separates industrial operations into multiple levels, including physical processes, control systems, manufacturing operations, and enterprise systems. This layered approach provides a clear separation of responsibilities and supports structured interaction between operational technology and information technology systems [1, 6].

The model has been widely adopted due to its ability to organize complex industrial systems and provide a common reference for system design. It has also influenced numerous industrial standards and integration approaches, making it a cornerstone of industrial automation architecture [7].

However, ISA-95 is primarily a conceptual model and does not prescribe specific implementation strategies. In practice, it is often implemented through vertical integration between adjacent layers, which introduces several limitations.

2.2.3 Limitations of Hierarchical Architectures

One of the main limitations of hierarchical architectures is the rigidity of communication structures. Systems are typically restricted to interacting with adjacent layers, which limits flexibility and increases the complexity of data flows [8].

This rigid structure results in increased latency, as data must traverse multiple layers before reaching its destination. It also introduces redundant data transformations and reduces system responsiveness, particularly in environments requiring real-time data processing [9].

Scalability is another significant challenge. As industrial systems expand, the number of required integrations grows rapidly, leading to increased engineering effort and maintenance complexity [3].

Data fragmentation is also a critical issue. In hierarchical architectures, data is distributed across multiple systems and layers, each with its own representation and storage model. This fragmentation limits system-wide visibility and makes it difficult to support advanced analytics and data-driven decision-making [10].

Furthermore, industrial environments are inherently heterogeneous, consisting of systems from different vendors using diverse communication protocols. Integrating these systems requires additional middleware and translation layers, increasing system complexity [11].

2.2.4 Transition Toward Data-Centric Architectures

In response to these limitations, industrial architectures have increasingly shifted toward data-centric and event-driven approaches. These architectures emphasize decoupled communication, real-time data sharing, and flexible integration between systems [12].

Instead of relying on strict hierarchical communication paths, data-centric architectures enable systems to interact through shared data models and asynchronous messaging mechanisms. This approach reduces coupling and improves scalability and interoperability [13].

The Unified Namespace paradigm represents a key development within this transition. By organizing industrial data into a shared and real-time namespace, UNS architectures provide a unified representation of system state and enable more efficient data integration across industrial environments [2].

2.3 Industrial Communication and Protocol Diversity

Industrial systems operate within highly heterogeneous communication environments, where a wide range of protocols coexist across different architectural layers. Field-level communication is typically dominated by protocols such as Modbus, PROFINET, EtherCAT, and CAN, which are designed for deterministic and real-time control. At higher levels, protocols such as OPC UA provide platform-independent communication and support structured information modeling [14, 15, 16].

This diversity reflects the historical evolution of industrial systems, where different protocols have been developed to meet specific requirements such as latency constraints, reliability, and vendor-specific implementations. However, the coexistence of these protocols introduces significant challenges for system integration, particularly in environments that require real-time data exchange and cross-system interoperability [17, 18].

To address these challenges, industrial systems often rely on middleware solutions and protocol translation mechanisms. While these approaches enable interoperability, they introduce additional complexity, increase latency, and reduce scalability. As industrial systems scale across multiple sites and platforms, these limitations become increasingly significant [19, 16].

OPC UA has emerged as a key standard for addressing interoperability challenges due to its support for structured data models and secure communication. It enables semantic representation of industrial data and provides a unified interface for accessing heterogeneous devices. However, its relatively high complexity and resource requirements can limit its effectiveness in large-scale data distribution scenarios [14, 15].

These limitations highlight the need for complementary communication paradigms capable of supporting scalable and flexible data integration.

2.4 MQTT and AMQP in Industrial Systems

2.4.1 MQTT and Telemetry Communication

MQTT is a lightweight publish–subscribe messaging protocol designed for efficient communication in distributed and resource-constrained environments. It has become one of the most widely adopted protocols in IIoT systems due to its simplicity, low bandwidth requirements, and asynchronous communication model [20, 21].

In industrial applications, MQTT is primarily used for telemetry communication, where continuous streams of data are transmitted from devices and sensors to higher-level systems. Its broker-based architecture allows multiple subscribers to access data streams independently, enabling scalable and decoupled system design [22, 19].

Experimental evaluations have demonstrated that MQTT performs efficiently in scenarios involving high message throughput and large numbers of clients, making it particularly suitable for real-time data dissemination in industrial environments [21].

However, MQTT is not designed for complex event processing or transactional messaging. It lacks advanced features such as message routing flexibility, guaranteed delivery mechanisms beyond basic quality-of-service levels, and support for workflow orchestration. These limitations restrict its applicability in event-driven and enterprise integration scenarios.

2.4.2 AMQP and Event-Driven Communication

AMQP is a protocol designed for reliable message-oriented communication in distributed systems. It provides advanced features such as message queuing, routing, acknowledg-

ments, and guaranteed delivery, making it well suited for implementing robust event-driven architectures [23, 24].

In industrial contexts, AMQP is particularly useful for handling discrete events such as alarms, system state changes, and workflow triggers. These events require reliable delivery and often involve coordination between multiple systems, which cannot be effectively handled using simple telemetry-based protocols [25, 26].

Compared to lightweight protocols such as MQTT, AMQP offers greater flexibility and reliability, particularly in scenarios involving complex messaging patterns and enterprise-level integration. Implementations such as RabbitMQ provide additional capabilities such as flexible routing and message persistence, further enhancing its applicability in industrial systems.

2.4.3 MQTT and AMQP as Complementary Paradigms

MQTT and AMQP represent two distinct communication paradigms that address complementary requirements in industrial systems. MQTT is optimized for continuous telemetry and real-time data distribution, while AMQP is designed for reliable event-driven communication and system coordination.

This distinction reflects the dual nature of industrial data. Continuous telemetry data represents the dynamic state of physical processes, while discrete events capture significant changes and triggers within the system.

Modern industrial architectures must support both paradigms simultaneously. However, integrating telemetry and event-driven communication within a single system introduces challenges related to data consistency, semantic alignment, and architectural complexity [18, 19].

Existing approaches often treat these paradigms separately, leading to fragmented architectures and increased integration effort. This fragmentation highlights the need for unified architectural models capable of integrating multiple communication paradigms within a consistent framework.

2.4.4 Message Semantics and Data Consistency

A critical distinction in industrial communication systems lies in the semantics of transmitted data, particularly the difference between state-based and event-based communication.

State-based communication, commonly associated with publish-subscribe systems, represents the current condition of a system through continuously updated values. This approach is well suited for telemetry and monitoring scenarios, where consumers rely on

the most recent data to infer system state [27, 28].

In contrast, event-based communication represents discrete occurrences that signify changes within the system. Events are typically associated with transitions, alarms, or workflow triggers and often require reliable delivery and ordering guarantees [29, 30].

The coexistence of these paradigms introduces challenges related to data consistency and synchronization. In distributed systems, discrepancies may arise due to delays, message loss, or differences in processing order, leading to inconsistent representations of system state across components [31, 32].

Ensuring consistency between state and event representations is therefore a key requirement for industrial architectures. This challenge is particularly relevant in environments where both continuous monitoring and discrete control actions must be supported simultaneously.

2.5 Implications for Industrial Integration

The coexistence of diverse communication protocols and paradigms presents a fundamental challenge in industrial system design. No single protocol is capable of addressing all integration requirements, particularly in large-scale and heterogeneous environments [17, 16].

As a result, hybrid architectures combining multiple communication technologies have become increasingly common. While these architectures provide flexibility, they also introduce additional complexity, particularly in relation to data consistency, interoperability, and system management [19].

A key requirement for modern industrial systems is therefore the ability to integrate multiple communication paradigms within a unified framework. Such a framework must support both telemetry and event-driven communication while ensuring consistent data representation and scalable system operation.

These requirements directly motivate the development of data-centric integration models such as Unified Namespace, which aim to provide a shared and consistent representation of system state across heterogeneous communication environments.

2.6 Unified Namespace

2.6.1 Concept and Architectural Principles

Unified Namespace (UNS) has emerged as a data-centric architectural paradigm aimed at addressing the limitations of traditional hierarchical integration models. In contrast

to layered architectures such as ISA-95, UNS organizes all system data into a shared and hierarchical namespace representing the real-time state of an industrial system [2].

Within this paradigm, all systems publish and consume data through a common namespace using publish–subscribe communication models. This enables decoupling between data producers and consumers, allowing systems to evolve independently while maintaining a consistent representation of system state.

The core principle of UNS is the concept of a “single source of truth,” where all relevant data is accessible through a unified structure. This approach reduces redundancy, simplifies integration, and enables real-time visibility across the entire system [33, 34].

UNS architectures are typically implemented using messaging technologies such as MQTT, where the namespace is represented as a hierarchical topic structure. This allows data to be organized according to functional, spatial, or logical relationships within the industrial environment.

2.6.2 Namespace Structure and Semantic Representation

The structure of the namespace is a critical factor in the effectiveness of Unified Namespace architectures. A well-defined namespace enables consistent data representation and facilitates interoperability between heterogeneous systems.

In industrial contexts, namespace structures are often aligned with existing standards such as ISA-95, where hierarchical levels represent enterprise, site, area, production line, and individual assets. This hierarchical organization supports intuitive data navigation and reflects the physical and logical structure of the industrial environment [35, 36].

Beyond structural organization, semantic consistency is essential for ensuring that data can be interpreted correctly across systems. Without shared semantics, identical data elements may be represented differently, leading to ambiguity and integration challenges.

To address this issue, recent research has explored the use of semantic models and ontologies for industrial data integration. Semantic approaches enable the enrichment of data with contextual meaning, supporting improved interoperability and enabling advanced functionalities such as reasoning and automated decision-making [37, 38].

However, the integration of semantic models into UNS architectures remains an open challenge, particularly in large-scale industrial systems where multiple stakeholders and legacy systems must be considered.

2.6.3 UNS in Industrial Practice

The adoption of UNS has been driven largely by industrial practitioners and has gained significant attention in the context of smart manufacturing and Industry 4.0. Practical implementations often focus on MQTT-based architectures, where devices and systems publish data into a shared namespace accessible to multiple consumers [2].

These implementations have demonstrated several advantages, including improved scalability, reduced integration effort, and enhanced system visibility. By eliminating point-to-point integrations, UNS simplifies system architecture and enables more flexible and dynamic interactions between components.

However, most existing implementations are based on informal design patterns rather than formalized architectural frameworks. This lack of standardization limits reproducibility and makes it difficult to evaluate and compare different implementations.

Furthermore, industrial implementations often focus primarily on telemetry data and do not fully address the integration of event-driven communication or enterprise-level workflows.

2.6.4 Data Flow and Communication Characteristics in UNS

A distinguishing feature of Unified Namespace architectures is the representation of system state as a continuous stream of updates rather than discrete transactional exchanges. This approach aligns with the publish-subscribe model, where changes in system state are propagated in real time to all interested consumers.

In this model, data flows are inherently asynchronous and event-driven, enabling systems to react to changes without requiring direct coupling between components. This supports scalability and flexibility, particularly in distributed environments where systems operate independently [36].

However, the distinction between state-based and event-based communication introduces challenges in system design. While telemetry data represents continuous system state, event-driven communication captures discrete occurrences such as alarms or process transitions.

Most existing UNS implementations focus primarily on telemetry and do not fully address the integration of event-driven communication. This limitation can lead to incomplete system representations and restrict the ability to implement complex workflows and coordination mechanisms.

The integration of both communication paradigms within a unified namespace remains an important research challenge and is a key focus of the proposed EirVah architecture.

2.6.5 Comparison with Alternative Integration Approaches

UNS can be viewed as part of a broader shift toward data-centric and event-driven architectures. Alternative approaches include service-oriented architectures (SOA), microservices, and event-driven architectures (EDA), each of which addresses different aspects of system integration [39, 40].

SOA emphasizes modularity and service reuse but often relies on tightly coupled interfaces and centralized orchestration. Microservices architectures improve scalability and flexibility but introduce additional complexity in service coordination and data consistency.

Event-driven architectures focus on asynchronous communication and decoupling between components, making them well suited for distributed systems. However, they do not inherently provide a unified representation of system state.

UNS differs from these approaches by combining data-centric design with real-time state representation. Instead of focusing solely on services or events, it provides a unified data model that integrates multiple communication paradigms within a shared namespace.

2.6.6 Scalability and Distribution in UNS Architectures

Scalability is a fundamental requirement for modern industrial systems, particularly in environments characterized by distributed production facilities and large numbers of connected devices. Unified Namespace architectures inherently support scalability through their decoupled communication model and hierarchical organization.

Distributed systems research has demonstrated that publish–subscribe architectures can scale effectively by decoupling producers and consumers and enabling asynchronous communication [27, 28]. This property aligns closely with the principles of UNS, where data is shared through a common namespace accessible to multiple participants.

In large-scale deployments, UNS architectures may span multiple brokers and infrastructure domains, enabling data sharing across sites, regions, and organizational boundaries. Such distributed configurations support multi-site manufacturing and facilitate the integration of geographically dispersed systems [41, 26].

However, scalability introduces additional challenges related to data consistency, synchronization, and system coordination. Ensuring that all components maintain a consistent view of system state becomes increasingly difficult as system size and complexity grow [31].

Cloud-native technologies provide mechanisms for addressing these challenges through dynamic scaling, fault tolerance, and distributed orchestration. Nevertheless, their inte-

gration with industrial data architectures remains an open research problem.

2.6.7 Limitations of Current UNS Approaches

Despite its potential, the current state of UNS exhibits several important limitations.

First, most implementations are strongly tied to MQTT, which limits their ability to support multiple communication paradigms. While MQTT is effective for telemetry, it does not fully address event-driven communication and enterprise integration requirements.

Second, there is a lack of formal architectural definitions. Existing UNS implementations are often described through informal guidelines or case studies rather than rigorous models, making it difficult to generalize and evaluate their effectiveness [2].

Third, deployment considerations are often neglected. Many implementations assume a specific infrastructure setup and do not address deployment across heterogeneous environments such as edge, private cloud, and public cloud systems.

Fourth, semantic consistency remains a challenge. While UNS provides a unified namespace, it does not inherently guarantee consistent data modeling across systems. This can lead to inconsistencies and reduce the effectiveness of the architecture.

Finally, cost considerations are rarely addressed in the literature. Many implementations rely on proprietary platforms or commercial solutions, which can limit adoption and scalability, particularly for small and medium-sized enterprises.

In addition, observability is not explicitly addressed in most UNS implementations. While the namespace provides visibility into system state, it does not inherently expose operational metrics related to system performance, communication latency, or resource utilization. As industrial systems become increasingly distributed and cloud-native, the ability to monitor and analyze system behavior becomes essential for ensuring reliability, scalability, and performance.

2.6.8 Emerging Directions and Research Opportunities

Recent research and industrial developments suggest several directions for advancing UNS architectures.

One important direction is the integration of multiple communication paradigms within a unified framework. By combining telemetry and event-driven communication, it is possible to create more flexible and expressive architectures capable of supporting a wider range of industrial applications.

Another direction is the incorporation of semantic modeling and contextualization mechanisms. This includes the use of ontologies and structured data models to ensure

consistency and improve interoperability across systems [42, 43].

Edge computing also plays an increasingly important role, enabling local data processing and reducing latency. The integration of edge processing with UNS architectures can improve system responsiveness and support decentralized decision-making.

Finally, there is growing interest in open and cost-efficient architectures based on open-source technologies. These approaches aim to reduce vendor dependency and make advanced industrial integration solutions more accessible.

These developments highlight the need for formal and scalable UNS architectures that can integrate multiple communication paradigms, support heterogeneous deployment environments, and provide cost-efficient solutions.

2.7 Implications for the Proposed Research

The analysis of Unified Namespace highlights both its potential and its current limitations. While UNS provides a promising foundation for industrial data integration, it remains insufficiently formalized and lacks support for key requirements such as multi-protocol integration, deployment flexibility, and cost efficiency.

These limitations directly motivate the development of the EirVah framework, which aims to extend UNS into a formal, scalable, and open reference architecture capable of addressing the challenges identified in this chapter.

2.8 Cyber-Physical Systems and Industrial Integration

Cyber-physical systems (CPS) represent a fundamental paradigm in modern industrial environments, integrating physical processes with computational and communication capabilities. CPS architectures enable real-time monitoring, control, and optimization by establishing continuous feedback loops between physical systems and digital components [44, 45].

A key characteristic of CPS is the emphasis on system observability and feedback-driven control. Data collected from physical processes is continuously analyzed and used to inform decision-making, enabling adaptive and autonomous system behavior. This aligns closely with principles of cybernetics, where systems are understood in terms of communication, feedback, and control [46, 47].

In industrial contexts, CPS architectures support a wide range of applications, including predictive maintenance, process optimization, and digital twins. These applications

rely on the ability to integrate data from multiple sources and maintain a consistent representation of system state across distributed environments [48, 49].

However, CPS integration remains challenging due to the heterogeneity of industrial systems and the need for real-time data exchange. These challenges highlight the importance of scalable and flexible integration architectures capable of supporting continuous data flows and system-wide coordination.

2.9 Edge Computing and Semantic Contextualization

Edge computing has emerged as a key enabler of modern CPS architectures by enabling data processing closer to the source of data generation. By performing computation at the edge, systems can reduce latency, improve responsiveness, and decrease reliance on centralized infrastructure [50, 51].

In industrial environments, edge systems are responsible for tasks such as data acquisition, protocol translation, filtering, and preprocessing. These functions are essential for integrating heterogeneous devices and ensuring that data is suitable for higher-level processing and analysis.

Recent research has explored the use of semantic contextualization at the edge, where raw data is enriched with meaning and structured according to predefined models. This enables more effective data integration and supports advanced applications such as digital twins and intelligent decision-making [52, 53].

Semantic reasoning at the edge has also been proposed as a mechanism for enabling localized intelligence and reducing the need for centralized processing. However, such approaches remain limited in practical deployment due to their complexity and the lack of integration with scalable industrial architectures.

As a result, semantic processing at the edge is best viewed as a supporting capability rather than a central architectural component.

2.10 Cloud-Native and Scalable Industrial Architectures

Cloud computing has become a fundamental component of modern industrial systems, enabling scalable data storage, processing, and analytics. More recently, cloud-native technologies such as containerization and orchestration platforms have further enhanced the scalability and flexibility of distributed systems [54, 55].

Kubernetes has emerged as a de facto standard for managing containerized applications, providing capabilities such as automated deployment, scaling, and fault tolerance. These features make it particularly suitable for industrial environments that require high availability and dynamic resource allocation [56].

In industrial contexts, cloud-native architectures enable the deployment of distributed services across multiple environments, including edge devices, private data centers, and public cloud platforms. This supports multi-site operations and facilitates the integration of geographically distributed systems.

However, the integration of cloud-native technologies with industrial systems presents several challenges, including latency constraints, security concerns, and the need for reliable communication across heterogeneous environments. These challenges highlight the importance of designing architectures that can operate consistently across edge, cloud, and hybrid deployments.

Hybrid cloud architectures have emerged as a practical solution for industrial environments that require both local processing and centralized coordination. By combining edge, private cloud, and public cloud resources, hybrid architectures enable flexible deployment strategies that can adapt to specific operational requirements [57, 58].

In industrial systems, hybrid deployments allow latency-sensitive and safety-critical operations to remain at the edge, while computationally intensive tasks such as data analytics and optimization can be performed in centralized cloud environments. This approach provides a balance between performance, scalability, and data governance [51].

However, hybrid environments introduce additional complexity, particularly in relation to data synchronization, security, and system orchestration. Ensuring consistent system behavior across heterogeneous infrastructure requires robust architectural design and reliable communication mechanisms [59].

These challenges highlight the importance of designing industrial architectures that are inherently cloud-agnostic and capable of operating seamlessly across edge, private, and public cloud environments.

2.11 Cost-Efficient and Open Architectures

Cost efficiency is a critical factor in industrial digitalization, particularly for small and medium-sized enterprises that may lack the resources to invest in large-scale proprietary solutions. Many existing industrial platforms rely on vendor-specific technologies, leading to high licensing costs and limited flexibility [60, 61].

Vendor lock-in is a significant concern, as it restricts the ability of organizations to adapt their systems and integrate new technologies. This can hinder innovation and limit

the scalability of industrial solutions.

Open architectures based on open-source technologies offer a promising alternative. By leveraging widely available tools and standards, such architectures can reduce costs, improve flexibility, and enable greater interoperability [62, 63].

Despite these advantages, there is limited research on how to design cost-efficient industrial architectures that also meet performance, scalability, and reliability requirements. This gap is particularly relevant in the context of Unified Namespace, where most existing implementations focus on functionality rather than economic considerations.

In addition to licensing costs, proprietary industrial platforms often introduce indirect costs related to system integration, maintenance, and vendor dependency. These costs can accumulate over time and significantly impact the total cost of ownership, particularly in long-lived industrial systems [64, 65].

Vendor lock-in is a major concern in industrial environments, as it restricts the ability of organizations to adapt their systems, integrate new technologies, or migrate between platforms. This limitation can hinder innovation and reduce long-term flexibility [66].

Open-source technologies offer an alternative approach by enabling organizations to build flexible and customizable systems without incurring high licensing fees. The use of open standards and community-driven tools also facilitates interoperability and reduces dependency on specific vendors [67, 68].

However, open architectures also introduce challenges, including the need for in-house expertise, integration effort, and operational complexity. Balancing cost efficiency with performance, reliability, and maintainability remains a key challenge in industrial system design.

2.12 Research Gap

The literature reviewed in this chapter highlights several important gaps in the current state of industrial integration architectures.

First, traditional hierarchical models such as ISA-95 are not well suited for modern industrial environments that require real-time data access, scalability, and flexible system integration.

Second, industrial communication remains highly fragmented, with diverse protocols and paradigms that are difficult to integrate into a unified framework.

Third, existing Unified Namespace implementations lack formal architectural definitions and are predominantly limited to telemetry-based communication. They do not fully address the integration of event-driven communication or enterprise-level workflows.

Fourth, deployment considerations are insufficiently addressed. There is a lack of architectures capable of operating consistently across edge, cloud, and hybrid environments.

Fifth, cost efficiency and openness are largely neglected in existing research, despite their importance for industrial adoption.

Finally, there is a limited integration of cyber-physical systems theory, cybernetics, and modern industrial architectures, particularly in relation to feedback, observability, and system coordination.

These gaps collectively motivate the development of the EirVah framework, which aims to provide a scalable, cost-efficient, and open reference architecture for Unified Namespace in Industrial IoT.

Chapter 3

Theoretical Framework

3.1 Introduction

This chapter establishes the theoretical foundations of the proposed research by situating it within the domains of cybernetics, cyber-physical systems (CPS), and data-centric architectures. While Chapter 2 examined the state of the art in industrial integration, this chapter focuses on the conceptual principles that underpin the design of scalable and adaptive industrial systems.

The proposed EirVah architecture is not only a technical framework but also a system-level construct grounded in feedback, observability, and control. These principles are central to cybernetics and provide a theoretical basis for understanding how complex industrial systems can be integrated, monitored, and optimized.

3.2 Cybernetics and System Theory

Cybernetics, originally introduced by Wiener, is concerned with the study of control and communication in systems, both biological and mechanical [46]. At its core, cybernetics emphasizes the role of feedback in enabling systems to regulate their behavior and maintain stability in the presence of external disturbances.

Closely related to cybernetics is general systems theory, which provides a broader framework for understanding complex systems and their interactions. Bertalanffy emphasized that systems should be understood as interconnected wholes rather than isolated components, highlighting the importance of structure, interaction, and organization [69].

A fundamental concept in cybernetics is the feedback loop, where information about the state of a system is continuously monitored and used to influence future behavior. This concept is directly applicable to industrial systems, where sensors, controllers, and actuators interact to maintain desired operating conditions.

Ashby further extended cybernetic theory by introducing the concept of variety and the law of requisite variety, which states that a control system must possess sufficient complexity to manage the variability of the system it controls [47]. This principle highlights the importance of scalable and flexible architectures capable of handling diverse and dynamic industrial environments.

3.3 Cyber-Physical Systems

Cyber-physical systems (CPS) build upon cybernetic principles by tightly integrating physical processes with computational and communication capabilities. These architectures enable continuous interaction between physical and digital components, supporting real-time monitoring, analysis, and control [44, 45, 70]. CPS are commonly conceptualized as consisting of three interconnected layers: the physical layer, the data layer, and the decision layer [71]. This layered abstraction serves as a foundational model for the design of the EirVah Framework, which is further developed and demonstrated throughout this project proposal.

In industrial environments, CPS are used to implement advanced capabilities such as predictive maintenance, process optimization, and digital twins. These applications rely on the ability to collect and process data from multiple sources and maintain a consistent representation of system state.

A key characteristic of CPS is the tight coupling between sensing, computation, and actuation. Data collected from physical systems is processed and used to generate control actions, which in turn influence the physical system. This creates a closed feedback loop that enables adaptive and autonomous behavior.

However, the integration of CPS components presents significant challenges, particularly in relation to scalability, interoperability, and data consistency. These challenges reinforce the need for architectural approaches that can support distributed and data-driven system integration.

3.4 Observability and System State Representation

Observability is a fundamental concept in both cybernetics and control theory, referring to the ability to infer the internal state of a system based on its outputs. In industrial systems, observability is achieved through sensors and data acquisition mechanisms that provide real-time information about system behavior.

From a control theory perspective, observability is essential for effective system regulation, as it enables accurate assessment of system conditions and supports informed

decision-making [72].

Modern industrial systems generate large volumes of data, but the availability of data alone does not guarantee effective observability. To achieve meaningful observability, data must be structured, contextualized, and accessible in a consistent manner.

This requirement highlights the importance of system state representation. A unified and coherent representation of system state enables improved monitoring, analysis, and decision-making. It also supports advanced applications such as digital twins and predictive analytics.

The concept of Unified Namespace can be interpreted as an architectural realization of system observability, where all relevant data is organized into a shared and accessible structure representing the current state of the system.

3.5 Feedback Loops and Control in Distributed Systems

Feedback loops are central to the operation of cyber-physical systems and industrial control systems. In traditional control systems, feedback is typically implemented at the device or process level. However, modern industrial systems require feedback mechanisms that operate across distributed and interconnected components.

Distributed systems theory provides additional insight into the challenges of implementing feedback in large-scale environments. Issues such as latency, synchronization, and consistency can significantly impact the effectiveness of feedback mechanisms [73].

Distributed feedback loops involve the coordination of multiple systems and data sources, enabling system-wide optimization and adaptive behavior. These loops rely on timely data exchange and consistent system state representation.

The design of effective feedback mechanisms therefore requires architectures that support reliable communication, low latency, and consistent data models. Unified Namespace architectures provide a foundation for such mechanisms by enabling real-time access to system state and supporting decoupled communication.

3.6 Data-Centric Architectures and System Integration

Traditional industrial architectures are often process-centric, focusing on specific workflows and control processes. In contrast, data-centric architectures prioritize the representation and accessibility of data as the primary means of system integration.

In data-centric architectures, systems interact through shared data models rather than direct interfaces. This approach reduces coupling and enables more flexible and scalable integration. It also aligns with cybernetic principles, where communication and information flow are central to system behavior.

Unified Namespace represents a data-centric approach to industrial integration, where all system data is organized into a shared namespace. This enables systems to interact indirectly through data, rather than through tightly coupled interfaces.

However, implementing data-centric architectures requires careful consideration of data modeling, consistency, and governance. Without a well-defined structure, data-centric systems can introduce new forms of complexity.

3.7 Towards a Formal Reference Architecture

The concepts discussed in this chapter provide the theoretical foundation for the proposed EirVah architecture. Cybernetics emphasizes feedback, control, and system behavior, while CPS highlights the integration of physical and digital components. Observability and data-centric design further define how system state can be represented and accessed.

Despite these advances, there is currently no formal reference architecture that integrates these concepts within the context of Unified Namespace and industrial IoT. Existing approaches remain fragmented and lack a unified theoretical framework.

The EirVah architecture aims to address this gap by providing a formal, scalable, and cost-efficient framework that integrates cybernetic principles, CPS architectures, and data-centric integration models.

This architecture will be presented in the following chapter.

Chapter 4

EirVah Architecture: A Scalable and Open Reference Model for Unified Namespace

4.1 Introduction

This chapter presents the proposed EirVah architecture, a scalable, cost-efficient, and open reference model for implementing Unified Namespace in industrial environments. Building on the limitations identified in Chapter 2 and the theoretical foundations established in Chapter 3, EirVah is designed to provide a formal and practical framework for industrial data integration.

The architecture integrates multiple communication paradigms, supports deployment across edge, cloud, and hybrid environments, and emphasizes observability, scalability, and openness. It is designed to be implemented using open-source technologies, enabling cost-efficient adoption while maintaining high performance and flexibility.

4.2 Abstract Architecture Foundation

The conceptual abstraction of CPS into physical, data, and decision layers provides a structured foundation for the architectural analysis of cyber-physical systems and serves as a core design principle of the EirVah architectural framework. The following diagram illustrates this layered abstraction:

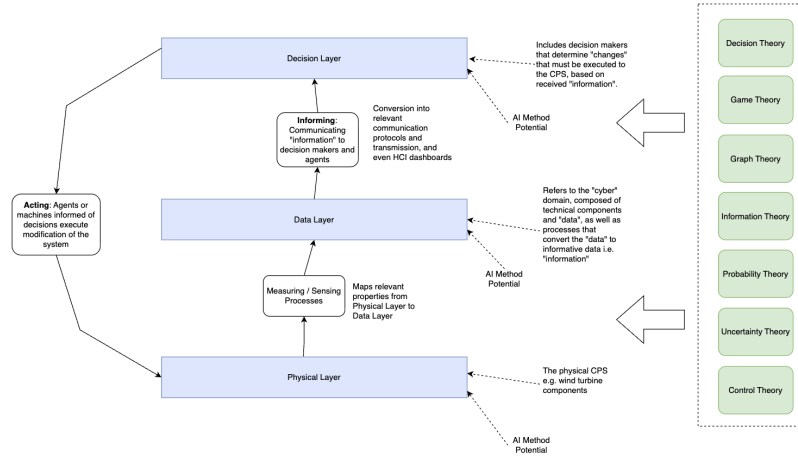


Figure 4.1: Abstract Representation of CPS

The main points of the diagram are as follows:

1. **Physical Layer.** The physical layer comprises tangible system components such as industrial machinery, embedded sensors, and actuators. It generates quantitative and qualitative operational data through continuous or event-driven measurements via sensing and measurement mechanisms. In some cases, edge-based artificial intelligence supports low-latency analysis. The layer executes control commands received from the decision layer via actuators to effect corresponding physical changes in the system.
2. **Data Layer.** The data layer manages storage, processing, and transformation of raw inputs into structured information. It includes databases, integration services, and enterprise systems. Processed data is contextualized and selectively forwarded from the physical layer to the decision layer. Artificial intelligence techniques may support cleansing, structuring, classification, and pattern detection.
3. **Decision Layer.** The decision layer evaluates processed information from the data layer to generate control actions based on rules, optimization methods, analytical models, or AI techniques. Decisions are transmitted to the physical layer for execution. A human-in-the-loop mechanism may supervise or override automated decisions, enabling collaborative intelligence in safety-critical or uncertain contexts.
4. **Applied Theory** From an abstract perspective, Cyber-Physical Systems (CPS) are underpinned by multiple theoretical domains that collectively support modeling, decision-making, communication, and control in complex, interconnected environments. Decision, Game, Graph, and Information Theory, together with Probability, Uncertainty, and Control Theory, can all be applied to describe system structure,

strategic interaction, data exchange, stochastic behavior, and feedback-driven dynamics within CPS.

4.3 Architectural Overview

The EirVah architecture is designed as a layered and distributed framework for integrating industrial devices, communication infrastructure, data processing, and system observability within a unified industrial data platform. The architecture combines principles from cyber-physical systems (CPS), distributed systems, and event-driven communication to support scalable, interoperable, and real-time industrial operations.

At the center of the architecture is the Unified Namespace (UNS), which provides a shared and continuously updated representation of system state across all connected components. This enables decoupled communication and consistent access to operational data throughout the system. The architecture further incorporates dedicated mechanisms for data ingestion, stream processing, storage, monitoring, and orchestration.

The framework is designed to operate across heterogeneous deployment environments, including edge devices, private cloud infrastructure, and public cloud platforms. This distributed deployment model aligns with modern architectural principles emphasizing scalability, interoperability, and loosely coupled communication [74].

The following diagram presents a high-level overview of the EirVah architecture:

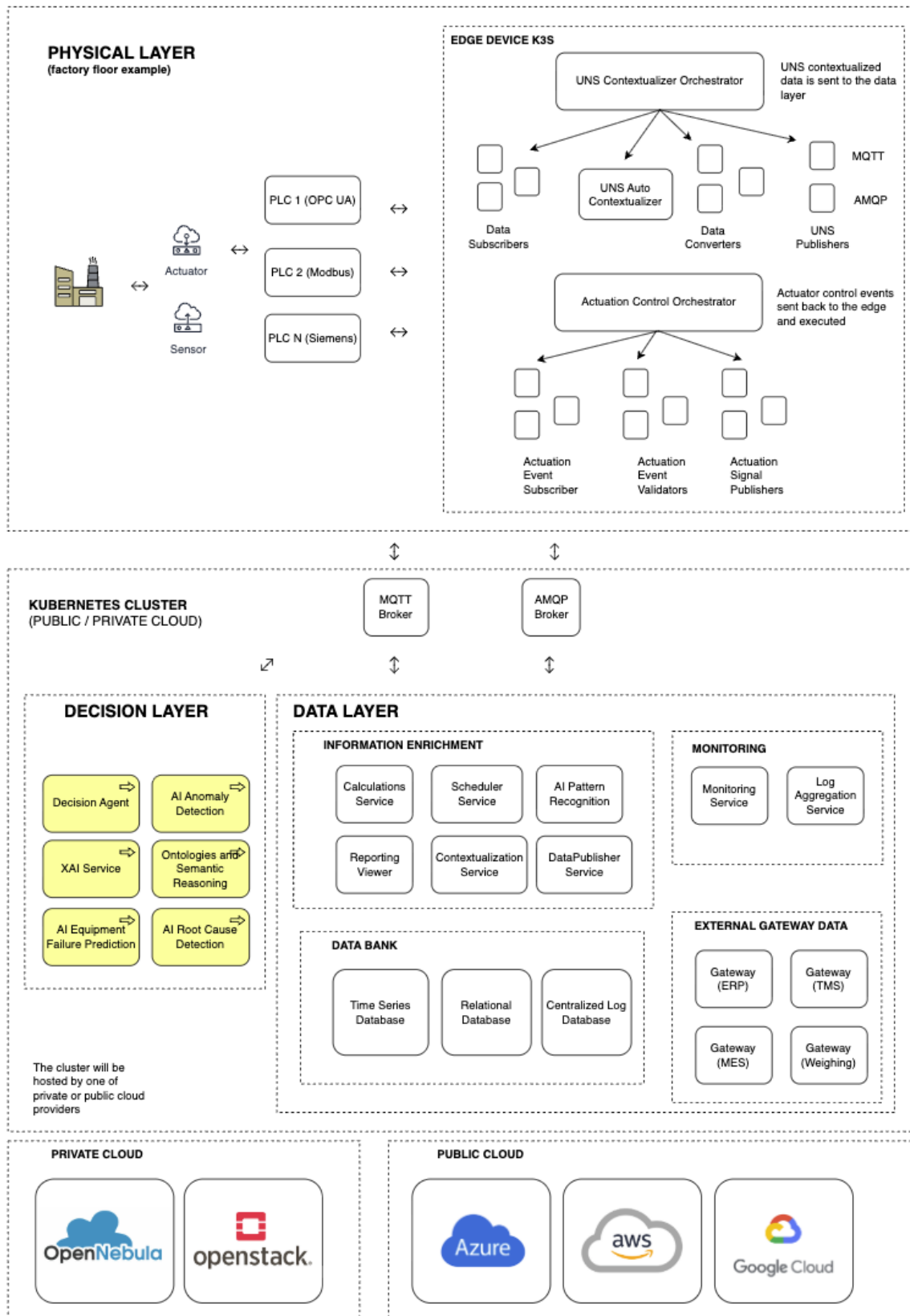


Figure 4.2: EirVah Framework Architecture Overview

4.4 Architectural Layers

The EirVah architecture is conceptually organized according to the CPS abstraction introduced earlier, consisting of interconnected physical, data, and decision layers. Building upon this foundation, the framework is divided into six primary architectural layers, each responsible for a distinct aspect of system functionality, communication, and data flow.

4.4.1 Edge Integration Layer

The Edge Integration Layer serves as the interface between the physical industrial environment and the higher-level data infrastructure of the EirVah framework. It is designed as a distributed microservices-based architecture responsible for acquiring, contextualizing, validating, and transmitting operational data from industrial devices and control systems.

As illustrated in Figure 4.2, the layer interfaces with heterogeneous industrial assets, including sensors, actuators, PLCs, and legacy automation systems using protocols such as OPC UA, Modbus, Siemens S7, and other proprietary interfaces. This abstraction enables interoperability across multi-vendor environments while minimizing direct protocol dependencies within the architecture.

A primary design objective of the layer is cost-effectiveness and accessibility. The framework is intentionally built around open-source technologies and lightweight infrastructure components to reduce reliance on expensive proprietary industrial platforms and vendor-specific middleware. Open standards and tooling such as Kubernetes/k3s, MQTT, AMQP, OPC UA integrations, and containerized microservices enable deployment on low-cost edge hardware while reducing operational costs, improving portability, and minimizing vendor lock-in.

The edge environment is deployed using lightweight Kubernetes distributions such as k3s, enabling containerized orchestration and lifecycle management of edge services. Each capability is implemented as an independent microservice, allowing services to be deployed, scaled, updated, and monitored independently. This microservices-oriented approach provides several architectural advantages:

- **Scalability.** Individual services such as protocol adapters, contextualizers, and publishers can scale independently depending on workload characteristics and device density.
- **Loose Coupling.** Services communicate through asynchronous messaging mechanisms, reducing direct interdependencies and improving architectural flexibility.

- **Fault Isolation.** Failures within a single edge service do not necessarily propagate across the system, improving operational resilience and maintainability.
- **Extensibility.** New industrial protocols, validation services, or processing components can be introduced without requiring modifications to the overall architecture.
- **Portability.** Containerized deployment enables consistent execution across edge gateways, on-premise infrastructure, and cloud environments.
- **Cost Efficiency.** Open-source microservices deployed on lightweight orchestration platforms significantly reduce infrastructure and licensing costs compared to traditional monolithic industrial integration solutions.

Within the edge environment, the architecture introduces two primary orchestration components:

- **UNS Contextualizer Orchestrator.** Responsible for transforming raw industrial signals into contextualized UNS-compliant events. This process includes protocol translation, data normalization, metadata enrichment, and publication into the Unified Namespace using messaging protocols such as MQTT and AMQP.
- **Actuation Control Orchestrator.** Responsible for handling outbound control flows from higher architectural layers back toward physical devices. This includes event validation, command routing, signal publishing, and actuator coordination.

The layer additionally supports modular edge microservices including:

- Data subscribers
- Data converters
- UNS publishers
- Actuation event subscribers
- Validation services
- Signal publishers

Together, these services enable scalable and decoupled communication between industrial assets and the broader CPS architecture. By performing contextualization, validation, and preprocessing directly at the edge, the framework reduces unnecessary upstream traffic, minimizes latency, and improves responsiveness for time-sensitive industrial operations.

Edge computing capabilities are particularly important in industrial environments where deterministic behavior, intermittent connectivity, and near real-time responsiveness must be maintained [75].

4.4.2 Unified Namespace and Messaging Layer

The Unified Namespace (UNS) and Messaging Layer forms the communication backbone of the EirVah architecture. It provides a shared, real-time representation of system state through which distributed services, edge nodes, databases, and analytical applications interact in a decoupled manner.

The UNS enables industrial data to be organized into a consistent and contextualized information space independent of the originating device, protocol, or subsystem. Rather than relying on tightly coupled point-to-point integrations, components communicate asynchronously through publish-subscribe messaging patterns, improving scalability, interoperability, and architectural flexibility.

MQTT is used as the primary telemetry protocol for lightweight and efficient distribution of operational data across edge and cloud environments. Its low overhead and publish-subscribe communication model make it well suited for industrial IoT and distributed CPS environments [76]. Lightweight brokers such as Mosquitto further support cost-effective deployment on constrained edge infrastructure.

In parallel, AMQP-based messaging supports event-driven workflows requiring reliable delivery, routing guarantees, and coordination between distributed services. Implementations such as RabbitMQ enable asynchronous orchestration of system events, actuation requests, and inter-service communication [77].

The coexistence of MQTT telemetry streams and AMQP event messaging aligns with modern distributed streaming architectures, where continuous system-state propagation and discrete event processing operate simultaneously [78].

4.4.3 Data and Persistence Layer

The Data and Persistence Layer is responsible for contextualization, enrichment, storage, and retrieval of operational and analytical data generated throughout the system.

As illustrated in Figure 4.2, the layer consists of several distributed microservices responsible for:

- Data contextualization
- Reporting and visualization support
- Scheduling and orchestration

- AI-driven pattern recognition
- Calculation and enrichment services
- Data publishing and transformation

These services operate on contextualized UNS data streams and provide higher-level information processing capabilities for downstream analytical and decision-making applications.

The persistence infrastructure combines multiple storage paradigms to support heterogeneous industrial workloads:

- **Time-series databases** for telemetry and sensor data
- **Relational databases** for transactional and structured information
- **Centralized log storage** for observability and system diagnostics

This multi-model storage approach enables scalable handling of high-frequency industrial data while supporting historical analysis, traceability, and integration with analytics pipelines. Distributed data architectures are essential for managing the scale and heterogeneity of modern industrial systems [79].

The layer additionally integrates external enterprise systems such as ERP, MES, TMS, and weighing systems through dedicated gateway services, enabling bidirectional information exchange between operational technology (OT) and enterprise information technology (IT) domains.

4.4.4 Decision and Analytics Layer

The Decision and Analytics Layer is responsible for transforming contextualized industrial data into actionable insights, intelligent recommendations, and operational decision support.

This layer contains analytical and AI-driven services that operate on both real-time and historical data streams obtained from the Unified Namespace and persistence infrastructure. Example capabilities within this layer include:

- AI-based anomaly detection
- Root-cause analysis
- Predictive maintenance and equipment failure prediction
- Explainable AI (XAI) services

- Ontology-based semantic reasoning
- Decision support and optimization services

These components enable intelligent monitoring, process optimization, and data-driven operational analysis across distributed industrial environments. By combining machine learning, semantic reasoning, and contextualized CPS data, the architecture supports advanced Industry 4.0 use cases while maintaining interoperability and scalability.

4.4.5 Observability Layer

The Observability Layer is responsible for monitoring system health, performance, reliability, and operational behavior across the distributed EirVah architecture.

Monitoring services continuously collect metrics, logs, and system telemetry from edge devices, messaging infrastructure, databases, and microservices. Metrics-based observability has become a fundamental practice for operating distributed systems reliably at scale [80].

Prometheus is used for metric collection and monitoring, while Grafana provides real-time visualization, dashboarding, and operational analysis capabilities. Centralized log aggregation services support troubleshooting, anomaly investigation, and performance analysis across distributed environments.

Beyond operational monitoring, observability also supports experimental evaluation and system optimization by enabling measurable analysis of throughput, latency, resource utilization, and fault behavior.

4.4.6 Orchestration and Deployment Layer

The Orchestration and Deployment Layer manages distributed deployment, scaling, and lifecycle management across edge, private cloud, and public cloud environments.

The architecture leverages Kubernetes-based orchestration platforms to provide container scheduling, automated deployment, service discovery, resilience, and fault recovery capabilities [81]. Lightweight Kubernetes distributions such as k3s are used at the edge, while full Kubernetes clusters can be deployed within private or public cloud infrastructure.

Containerization technologies enable consistent execution across heterogeneous environments, improving portability, scalability, and maintainability [82]. This deployment model supports hybrid-cloud CPS architectures capable of distributing workloads dynamically between edge and cloud resources.

The framework is intentionally designed around open-source infrastructure technologies including Kubernetes, OpenStack, OpenNebula, MQTT brokers, and containerized microservices. This reduces dependency on expensive proprietary industrial platforms while enabling cost-effective deployment and extensibility for organizations of varying scale.

4.5 Data Flow and System Interaction

Data flow within the EirVah architecture follows a distributed publish–subscribe communication model centered around the Unified Namespace (UNS). Industrial data originates at the physical and edge layers, where sensors, PLCs, and other industrial devices generate operational events and telemetry streams. These data streams are acquired and contextualized by edge microservices before being published into the messaging infrastructure.

Once published, data becomes available to all authorized system components through asynchronous messaging mechanisms such as MQTT and AMQP. This enables analytical services, persistence systems, monitoring platforms, and decision-support applications to consume data independently without requiring direct point-to-point integrations.

The architecture supports both continuous telemetry streams and discrete event-driven interactions. Telemetry data is primarily used for real-time monitoring, visualization, and historical persistence, while event-driven messaging enables orchestration workflows, actuation requests, anomaly notifications, and inter-service coordination.

A key characteristic of the architecture is the decoupling of data producers and consumers through the Unified Namespace. Services communicate indirectly through shared contextualized data models rather than explicit service dependencies. This reduces architectural rigidity, simplifies integration of new services, and enables components to evolve independently over time.

The distributed communication model additionally improves scalability and fault isolation. Edge nodes can continue local processing independently, while downstream cloud services can scale horizontally depending on workload requirements. The asynchronous publish–subscribe approach further reduces communication bottlenecks and improves responsiveness in large-scale industrial environments [83].

Figure 4.2 illustrates the primary system interaction flow across the physical, edge, data, and decision layers of the EirVah framework.

4.6 Deployment Model

The EirVah architecture supports edge, cloud, and hybrid deployment models, enabling flexible distribution of computation, storage, and communication across industrial environments.

Edge deployments provide low-latency processing and localized decision-making capabilities close to physical devices, while cloud environments provide scalable infrastructure for large-scale data storage, analytics, orchestration, and AI-driven services. Hybrid deployment models combine these capabilities, allowing workloads to be distributed dynamically according to performance, scalability, reliability, and data-governance requirements.

This deployment flexibility aligns with modern distributed and cloud computing paradigms, where computation is increasingly decentralized across edge and cloud infrastructure to support scalable and responsive cyber-physical systems [58].

4.7 Key Architectural Properties

The EirVah architecture is designed around several core architectural properties that support scalable, interoperable, and cost-effective industrial cyber-physical systems:

- **Scalability:** Achieved through distributed microservices, asynchronous publish-subscribe messaging, and container orchestration platforms capable of horizontal scaling across edge and cloud environments.
- **Interoperability:** Enabled through unified data representation, protocol abstraction, and integration support for heterogeneous industrial systems and communication standards.
- **Observability:** Supported through centralized monitoring, metrics collection, log aggregation, and real-time visualization services that provide visibility into system behavior and performance.
- **Fault Tolerance and Resilience:** Distributed services and decoupled communication mechanisms improve fault isolation and enable continued operation despite localized failures or intermittent connectivity.
- **Cost Efficiency:** Achieved through the use of open-source technologies, lightweight infrastructure platforms, and commodity hardware, reducing dependency on proprietary industrial software stacks.

- **Flexibility and Extensibility:** Supported through modular microservice design and hybrid deployment capabilities, enabling the architecture to adapt to evolving operational requirements and integrate new services incrementally.
- **Portability:** Containerized deployment enables consistent execution across edge devices, private cloud infrastructure, and public cloud environments.

4.8 Summary

This chapter presented the EirVah architecture as a scalable and distributed reference framework for Unified Namespace (UNS)-based industrial IoT and cyber-physical systems. The architecture integrates heterogeneous industrial devices, distributed messaging infrastructure, data processing services, observability mechanisms, and AI-driven analytical components within a unified and interoperable framework.

The proposed architecture emphasizes scalability, interoperability, extensibility, observability, and cost efficiency through the use of distributed microservices, asynchronous messaging patterns, hybrid deployment models, and open-source technologies.

The following chapter presents the research methodology and evaluation approach used to validate and assess the proposed architecture.

Chapter 5

Research Methodology

5.1 Introduction

This chapter describes the methodology used to design, implement, and evaluate the proposed EirVah architecture. The objective of this research is to develop and validate a scalable, cost-efficient, and open reference architecture for Unified Namespace in industrial IoT.

The methodology combines system design, practical implementation, and experimental evaluation. A significant portion of the architecture will be implemented using real technologies, while selected components of the industrial environment will be simulated to enable controlled experimentation.

5.2 Research Approach

The research follows a design science methodology, where the primary objective is the creation and evaluation of an artefact—in this case, the EirVah architecture.

The approach consists of the following stages:

1. **Problem Identification:** Identification of limitations in current industrial integration architectures.
2. **Architecture Design:** Development of the EirVah reference architecture.
3. **Implementation:** Construction of a working system using open-source technologies.
4. **Evaluation:** Experimental validation of system performance, scalability, and cost efficiency.

This approach ensures that the research is both theoretically grounded and practically validated.

5.3 System Implementation Strategy

The EirVah architecture will be implemented as a distributed system consisting of multiple components deployed across edge and cloud environments.

Most components of the system will be implemented using real technologies, including:

- MQTT broker (e.g., Mosquitto)
- AMQP broker (RabbitMQ)
- Edge processing nodes (k3s)
- Cloud orchestration (Kubernetes)
- Data storage (InfluxDB, PostgreSQL)
- Observability stack (Prometheus, Grafana)

The industrial environment will be partially simulated. Specifically, shop-floor devices will be emulated by generating MQTT telemetry and AMQP event streams. This allows for controlled experimentation while maintaining a realistic system architecture.

This hybrid approach enables the evaluation of a near-real system while retaining experimental flexibility.

5.4 Infrastructure as Code and Reproducibility

To ensure reproducibility and consistency, the entire system will be defined and deployed using Infrastructure as Code (IaC) principles.

Deployment configurations will be managed using declarative definitions, enabling automated provisioning of infrastructure and services. This approach ensures that experiments can be repeated under identical conditions and supports systematic evaluation.

Reproducibility is a key requirement in scientific research and is particularly important in distributed systems, where configuration differences can significantly impact results.

5.5 Experimental Design

The evaluation of the EirVah architecture will be based on a series of controlled experiments designed to assess key system properties.

The experiments will focus on the following aspects:

- **Scalability:** System performance under increasing load.
- **Latency:** End-to-end delay in data propagation.
- **Throughput:** Volume of messages processed per unit time.
- **Reliability:** System behavior under failure conditions.
- **Resource Utilization:** CPU, memory, and network usage.
- **Cost Efficiency:** Resource consumption relative to system performance.

Experiments will be conducted across different deployment scenarios, including edge-only, cloud-only, and hybrid configurations.

5.6 Observability and Metrics Collection

Observability plays a central role in the evaluation of the EirVah architecture.

Prometheus will be used to collect metrics from all system components, including messaging brokers, edge nodes, orchestration platforms, and data storage systems. These metrics will include:

- Message throughput
- Message latency
- Queue depth
- System resource utilization
- Error rates and failure events

Grafana will be used to visualize metrics and support real-time analysis.

The observability layer enables precise measurement of system behavior and provides the data required for experimental evaluation. It also supports debugging and performance optimization.

5.7 Validation Strategy

The validity of the proposed architecture will be assessed through both quantitative and qualitative evaluation.

Quantitative evaluation will be based on the metrics collected during experiments, allowing for objective comparison of system performance under different conditions.

Qualitative evaluation will consider factors such as system flexibility, ease of integration, and architectural clarity.

The results will be compared against existing approaches where possible, highlighting improvements in scalability, cost efficiency, and system integration.

5.8 Limitations of the Methodology

While the proposed methodology enables comprehensive evaluation, it has certain limitations.

The use of simulated industrial data may not fully capture the complexity of real-world environments. However, it allows for controlled experimentation and repeatability.

Additionally, the evaluation focuses on a specific implementation of the architecture, which may not represent all possible deployment scenarios.

These limitations will be acknowledged and considered in the interpretation of results.

5.9 Summary

This chapter outlined the methodology used to design, implement, and evaluate the EirVah architecture. The approach combines real system implementation with controlled simulation, supported by observability-driven measurement and reproducible deployment.

The next chapter presents the evaluation plan and expected outcomes of the proposed research.

Chapter 6

Evaluation Plan and Expected Contributions

6.1 Introduction

This chapter outlines the evaluation plan for the proposed EirVah architecture and defines the expected contributions of the research. The evaluation focuses on validating the architecture in terms of scalability, performance, cost efficiency, flexibility, and reproducibility across different deployment environments.

In addition, this chapter positions the research within the broader academic and industrial context by identifying its scientific and practical contributions.

6.2 Evaluation Objectives

The evaluation of the EirVah architecture is designed to address the following research objectives:

- To assess the scalability of the architecture under increasing system load.
- To evaluate the performance of the architecture in terms of latency and throughput.
- To validate the integration of telemetry and event-driven communication within a unified framework.
- To assess the ability of the architecture to operate consistently across edge, cloud, and hybrid deployment models.
- To evaluate the cost efficiency of the architecture based on resource utilization and open-source implementation.

- To assess the reproducibility of the architecture through automated deployment and configuration using Infrastructure as Code.

These objectives directly correspond to the limitations identified in Chapter 2 and the design goals defined in Chapter 4.

6.3 Experimental Scenarios

The evaluation will be conducted across multiple experimental scenarios designed to reflect realistic industrial environments.

6.3.1 Baseline Scenario

The baseline scenario represents a simplified industrial system with a limited number of devices and data streams. This scenario is used to establish reference performance metrics for the architecture.

6.3.2 Scalability Scenario

In the scalability scenario, the number of simulated devices and message rates will be gradually increased. This allows evaluation of how the architecture behaves under increasing load and identifies potential bottlenecks.

6.3.3 Failure and Recovery Scenario

The architecture will be tested under failure conditions, such as broker outages or node failures. The objective is to evaluate system resilience and recovery behavior.

6.4 Deployment Model Evaluation

A key objective of the evaluation is to assess the ability of the EirVah architecture to operate consistently across different deployment models, including edge, cloud, and hybrid environments.

Rather than treating these environments as separate system configurations, the evaluation focuses on the architectural consistency of the framework when deployed across different infrastructure contexts. This reflects the design goal of EirVah as a cloud-agnostic and deployment-flexible architecture.

Experiments will therefore compare system behavior under the following deployment conditions:

- **Edge Deployment:** Processing and messaging components deployed locally using lightweight orchestration (e.g., k3s).
- **Cloud Deployment:** Centralized deployment using Kubernetes-based infrastructure.
- **Hybrid Deployment:** Distribution of components across edge and cloud environments.

The evaluation will assess how deployment context influences system performance, including latency, throughput, and resource utilization, while maintaining a consistent architectural model.

System performance across these deployment models will be evaluated using the metrics defined in Section 6.5.

This approach emphasizes that EirVah is not designed for a single deployment paradigm but rather provides a unified framework capable of operating seamlessly across multiple environments.

6.5 Evaluation Metrics

The evaluation will be based on quantitative metrics collected through the observability layer. These metrics are applied consistently across all experimental scenarios and deployment models, enabling objective comparison of system behavior under different conditions.

In particular, the metrics support evaluation of scalability, deployment flexibility, cost efficiency, and reproducibility by providing a unified measurement framework across edge, cloud, and hybrid environments.

- **Latency:** End-to-end delay between data generation and consumption.
- **Throughput:** Number of messages processed per second.
- **Resource Utilization:** CPU, memory, and network usage across system components.
- **System Availability:** Uptime and recovery time after failures.
- **Message Reliability:** Rate of successful message delivery.
- **Cost Efficiency:** Resource usage relative to system performance.

- **Reproducibility:** Ability to recreate identical deployments and experimental conditions across environments.

These metrics provide a comprehensive view of system performance and enable objective comparison between different configurations.

6.6 Cost Evaluation and Open-Source Feasibility

A key objective of this research is to demonstrate that scalable industrial architectures can be implemented in a cost-efficient manner using open-source technologies. In contrast to proprietary industrial platforms, which often involve significant licensing and infrastructure costs, the EirVah architecture is designed to operate using widely available open-source components.

The cost evaluation builds directly on the metrics defined in the previous section, using resource utilization measurements to estimate infrastructure and operational costs.

6.6.1 Cost Model

The cost model is based on the following components:

- **Compute Resources:** CPU and memory usage of edge and cloud nodes.
- **Storage:** Data storage requirements for telemetry and historical data.
- **Networking:** Data transfer between edge, cloud, and messaging systems.
- **Operational Overhead:** Deployment and maintenance effort.

Since the architecture is based on open-source technologies, licensing costs are assumed to be negligible.

6.6.2 Experimental Cost Measurement

Cost will be estimated indirectly through resource utilization metrics collected via the observability layer. Prometheus metrics will be used to measure CPU usage, memory consumption, network bandwidth, and storage requirements across system components.

These metrics will be mapped to equivalent infrastructure costs using standard cloud pricing models, enabling estimation of total cost of ownership under different deployment scenarios.

6.6.3 Open-Source Stack Evaluation

The architecture is implemented using open-source technologies, including MQTT brokers, RabbitMQ, Kubernetes, and open-source databases. This enables a significant reduction in licensing costs compared to proprietary industrial platforms.

The evaluation will assess whether the open-source stack can meet performance and scalability requirements without introducing excessive operational complexity.

6.6.4 Cost-Performance Trade-offs

An important aspect of the evaluation is the relationship between cost and performance. Increasing system performance typically requires additional computational resources, which can increase operational costs.

The experiments will analyze how latency, throughput, and reliability scale relative to resource consumption. This allows identification of configurations that provide optimal performance while minimizing cost.

6.6.5 Expected Outcome

It is expected that the EirVah architecture will demonstrate that high-performance industrial integration can be achieved using cost-efficient and open technologies.

This would provide strong evidence that advanced industrial architectures are accessible not only to large enterprises but also to small and medium-sized organizations.

6.7 Reproducibility and Infrastructure as Code

A central requirement of the proposed research is that the EirVah architecture should be reproducible across different deployment environments. In this context, reproducibility refers to the ability to recreate the same architectural configuration, system behavior, and experimental setup in a consistent and automated manner.

To achieve this, the framework will be deployed using Infrastructure as Code (IaC) principles. Infrastructure resources, orchestration settings, messaging components, storage services, and observability tools will be defined through declarative configuration files and deployment scripts. This enables automated provisioning of the full framework across edge, cloud, and hybrid environments.

The use of IaC supports reproducibility in two key ways. First, it ensures that the same system configuration can be deployed repeatedly with minimal manual intervention. Second, it allows experimental scenarios to be reconstructed under controlled and consistent conditions.

Reproducibility will therefore be evaluated not only in terms of successful redeployment, but also in terms of whether equivalent system performance and behavior can be observed across repeated deployments.

From a scientific perspective, this strengthens the validity of the research by enabling repeatability of experiments. From a practical perspective, it demonstrates that the architecture can be reliably deployed and maintained in real industrial environments.

6.8 Comparative Evaluation

Where possible, the EirVah architecture will be compared with existing integration approaches, including:

- Traditional hierarchical architectures based on ISA-95.
- MQTT-only Unified Namespace implementations.
- Event-driven architectures without a unified data model.

The comparison will focus on scalability, flexibility, system complexity, and cost efficiency.

6.9 Expected Results

The proposed architecture is expected to demonstrate several improvements over existing approaches.

- Improved scalability through distributed messaging and orchestration.
- Enhanced flexibility through integration of telemetry and event-driven communication.
- Reduced latency in edge and hybrid deployments.
- Improved observability through metrics-based monitoring.
- Significant cost reduction through the use of open-source technologies.
- High reproducibility through Infrastructure as Code and automated deployment.

6.10 Scientific Contributions

The research is expected to contribute to the field of industrial IoT and system integration in several ways.

6.10.1 Formalization of Unified Namespace

The research will provide a formal reference architecture for Unified Namespace, addressing the lack of standardized models in current literature.

6.10.2 Integration of Communication Paradigms

The architecture demonstrates how telemetry and event-driven communication can co-exist within a unified system.

6.10.3 Observability-Driven Architecture

The research introduces observability as a core architectural component, enabling measurement, optimization, and feedback-driven system behavior.

6.10.4 Deployment-Agnostic and Reproducible Architecture

The research contributes a framework capable of operating consistently across edge, cloud, and hybrid environments, while also enabling reproducible deployment through Infrastructure as Code.

6.10.5 Cost-Efficient Open Architecture

The research demonstrates that industrial integration can be achieved using open-source technologies without sacrificing performance.

6.11 Practical Contributions

The proposed architecture has practical implications for industrial organizations.

It provides a blueprint for implementing scalable, flexible, and reproducible industrial systems, reduces dependency on proprietary platforms, and enables data-driven decision-making.

The use of open technologies also supports wider adoption of Industry 4.0 principles, particularly among small and medium-sized enterprises.

6.12 Summary

This chapter presented the evaluation plan and expected contributions of the proposed EirVah architecture. The evaluation framework defines how the architecture will be as-

sessed, while the identified contributions highlight its relevance to both academic research and industrial practice.

Together, these elements complete the proposed research framework and establish a clear path for the execution and validation of the PhD project.

Chapter 7

Dissertation Theses and Research Objectives

7.1 Introduction

This chapter summarizes the core scientific direction of the proposed dissertation by defining the primary thesis statements together with the associated theoretical and practical objectives. These elements establish the relationship between the conceptual foundations of the research, the proposed EirVah architecture, and the experimental validation methodology.

The dissertation is positioned within the domains of industrial IoT, cyber-physical systems, distributed systems, and data-centric industrial integration. Its objective is not only to design an architectural framework, but also to formally investigate scalable and observable industrial integration through theoretical analysis and experimental evaluation.

7.2 Dissertation Thesis Statements

The proposed dissertation is based on the following thesis statements:

1. A formalized and observability-driven Unified Namespace architecture can improve scalability, interoperability, and deployment flexibility in distributed industrial IoT and cyber-physical systems.
2. The integration of telemetry-based and event-driven communication within a Unified Namespace architecture enables scalable and decoupled industrial system integration.

3. Distributed microservices and cloud-native deployment models enable deployment-agnostic industrial architectures capable of operating across edge, cloud, and hybrid environments.
4. Open-source and data-centric industrial architectures can provide cost-efficient alternatives to proprietary industrial integration platforms without compromising scalability or flexibility.
5. Observability-driven and Infrastructure-as-Code-based approaches improve reproducibility, system monitoring, and operational management of distributed industrial systems.

7.3 Theoretical Objectives

To satisfy the proposed dissertation theses, the following theoretical objectives must be addressed:

- To formalize Unified Namespace as a scalable and interoperable architectural model for industrial IoT and cyber-physical systems.
- To define a unified integration model combining telemetry-based and event-driven communication paradigms.
- To investigate system-state representation, observability, and feedback mechanisms in distributed industrial architectures.
- To apply principles of cybernetics, distributed systems, and cloud-native architectures to industrial data integration.
- To investigate scalability and communication efficiency in distributed industrial publish–subscribe architectures.
- To evaluate cost-efficiency trade-offs between open-source and proprietary industrial integration approaches.

7.4 Practical Objectives

To experimentally validate the proposed architecture, the following practical objectives will be pursued:

- To implement a working prototype of the EirVah architecture using open-source technologies.

- To integrate MQTT telemetry communication and AMQP event-driven messaging within a unified namespace framework.
- To deploy the architecture across edge, cloud, and hybrid environments using Kubernetes-based orchestration.
- To simulate industrial telemetry and event streams for controlled experimental evaluation.
- To implement observability and monitoring infrastructure using Prometheus and Grafana.
- To evaluate scalability, deployment flexibility, and cost efficiency of the proposed architecture across distributed environments.
- To ensure reproducibility through Infrastructure as Code and automated deployment mechanisms.

7.5 Current Research Outputs

The research conducted within this dissertation has already resulted in multiple scientific outputs related to industrial integration architectures, Unified Namespace, and event-driven industrial systems.

1. Primary author, “EirVah: An Open-Source Hybrid-Cloud Framework for Industrial Cyber-Physical Systems,” Springer Conference Proceedings, 2026. (In Press)
2. Primary author, “Edge-Based Industrial Data Contextualization for Unified Namespace Cyber-Physical Systems,” ELEKTRO 2026. (Submitted)
3. Co-author, “Integrating Digital Twins and Deep Reinforcement Learning for Urban Traffic Control,” ELEKTRO 2026. (Submitted)

In addition to scientific publications, the research has produced initial architectural designs, deployment configurations, and prototype implementations related to the EirVah framework, including Kubernetes-based orchestration, distributed messaging infrastructure, observability integration, and edge-based industrial communication services.

7.6 Summary

This chapter defined the primary dissertation thesis statements together with the associated theoretical and practical objectives of the proposed research. These elements establish the scientific scope of the dissertation and provide a structured foundation for the continued development, implementation, and evaluation of the EirVah architecture.

The following chapter concludes the proposal and summarizes the overall contribution and expected impact of the research.

Chapter 8

Conclusion

This proposal presented EirVah, a scalable, cost-efficient, and open reference architecture for Unified Namespace in industrial IoT. The proposed research addresses key limitations in current industrial integration approaches, including insufficient formalization of Unified Namespace, weak support for multi-paradigm communication, limited deployment flexibility, and the lack of cost-efficient open implementations.

The proposal established the academic and practical motivation for the research through a state-of-the-art review and a theoretical framework grounded in cybernetics, cyber-physical systems, and data-centric integration. It then defined the EirVah architecture as a deployment-agnostic and observable framework capable of operating across edge, cloud, and hybrid environments.

A methodology based on design science, near-real system implementation, observability-driven evaluation, and Infrastructure as Code was proposed to validate the architecture. The evaluation plan further defined how scalability, performance, reproducibility, and cost efficiency will be assessed.

Overall, the proposed research aims to contribute both a formal architectural model and an experimentally validated implementation strategy for Unified Namespace in industrial systems. In doing so, it seeks to advance the state of the art in industrial IoT integration while providing a practical and accessible framework for real-world adoption.

Bibliography

- [1] ISA. Enterprise-control system integration, 2000.
- [2] Juliam A. Díaz et al. Unified namespace: A new paradigm for industrial iot data integration. *IEEE Internet of Things Magazine*, 2025.
- [3] L. Chettri and R. Bera. A comprehensive survey on internet of things (iot) toward 5g wireless systems. *IEEE Internet of Things Journal*, 7(1):16–32, 2020.
- [4] S. D. Milić and B. M. Babić. Toward the future—upgrading existing remote monitoring concepts to iiot concepts. *IEEE Internet of Things Journal*, 7(12):11693–11700, 2020.
- [5] Henning Kagermann, Wolfgang Wahlster, and Johannes Helbig. Recommendations for implementing the strategic initiative industrie 4.0. *Final Report of the Industrie 4.0 Working Group*, 2013.
- [6] M. Singh, R. Goyat, and R. Panwar. Fundamental pillars for industry 4.0 development. *The TQM Journal*, 2024.
- [7] L. Apiliogullari. Digital transformation in project-based manufacturing. *International Journal of Production Economics*, 2022.
- [8] Sebastian Bader et al. Increasing the adaptability of manufacturing systems by data-centric communication. In *IEEE Emerging Technology and Factory Automation*, 2015.
- [9] Yang Lu. Industry 4.0: A survey on technologies, applications and open research issues. *Journal of Industrial Information Integration*, 2017.
- [10] Fei Tao, He Zhang, Ang Liu, and A. Y. C. Nee. Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics*, 2019.
- [11] Michael Weyrich and Christof Ebert. Reference architectures for the internet of things. *IEEE Software*, 2016.

- [12] Tim Teriete, Markus Böhm, Brandon K. Sai, Klaus Erlach, and Thomas Bauernhansl. Event-based framework for digitalization of value stream mapping. *Procedia CIRP*, 2022.
- [13] Petri Kannisto, David Hästbacka, Teresa Gutiérrez, Olli Suominen, Matti Vilkkö, and Peter Craamer. Plant-wide interoperability and decoupled, data-driven process control with message bus communication. *Journal of Industrial Information Integration*, 2022.
- [14] B. Galloway and G. Hancke. Introduction to industrial control networks. *IEEE Communications Surveys & Tutorials*, 2013.
- [15] W. Mahnke, S.-H. Leitner, and M. Damm. *OPC Unified Architecture*. Springer, 2009.
- [16] J. da Silva et al. Industrial iot systems: A survey. *IEEE Transactions on Industrial Informatics*, 2014.
- [17] L. Atzori, A. Iera, and G. Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010.
- [18] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi. Internet of things for smart cities. *IEEE Internet of Things Journal*, 1(1):22–32, 2014.
- [19] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao. A survey on internet of things: Architecture, enabling technologies, security and privacy. *IEEE Internet of Things Journal*, 4(5):1125–1142, 2017.
- [20] A. Banks and R. Gupta. Mqtt version 3.1.1. *OASIS Standard*, 2014.
- [21] D. Thangavel et al. Performance evaluation of mqtt and coap. *IEEE Conference*, 2014.
- [22] A. Al-Fuqaha et al. Internet of things: A survey. *IEEE Communications Surveys & Tutorials*, 2015.
- [23] OASIS. Amqp version 1.0. *OASIS Standard*, 2012.
- [24] S. Vinoski. Advanced message queuing protocol. *IEEE Internet Computing*, 2006.
- [25] H. Happel and S. Seedorf. Applications of event-driven architectures. *IEEE Software*, 2017.
- [26] J. Kreps et al. Kafka: A distributed messaging system. *NetDB*, 2011.

- [27] P. T. Eugster, P. A. Felber, R. Guerraoui, and A. M. Kermarrec. The many faces of publish/subscribe. *ACM Computing Surveys*, 2003.
- [28] A. Carzaniga, D. S. Rosenblum, and A. L. Wolf. Design and evaluation of a wide-area event notification service. *ACM Transactions on Computer Systems*, 2001.
- [29] B. Michelson. Event-driven architecture overview. *Patricia Seybold Group*, 2006.
- [30] A. Hinze, K. Sachs, and A. Buchmann. Event-based applications and enabling technologies. *Proceedings of the IEEE*, 2009.
- [31] W. Vogels. Eventually consistent. *Communications of the ACM*, 2009.
- [32] M. Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [33] M. Schultz. Unified namespace in smart manufacturing. *Manufacturing Letters*, 2020.
- [34] J. Parrish. Implementing unified namespace for industrial systems. *Industrial Engineering Journal*, 2021.
- [35] S. R. Bader et al. Reference architectures for industry 4.0. *Future Internet*, 2021.
- [36] D. Schmidt et al. Industrial iot and digital transformation: A review. *IEEE Access*, 2020.
- [37] I. Grangel-González et al. Semantic interoperability in industry 4.0. *IEEE Transactions on Industrial Informatics*, 2016.
- [38] S. Bader et al. Semantic models for industry 4.0. *IEEE Access*, 2021.
- [39] G. Hohpe and B. Woolf. *Enterprise Integration Patterns*. Addison-Wesley, 2004.
- [40] C. Richardson. *Microservices Patterns*. Manning, 2018.
- [41] M. Stonebraker. The end of an architectural era. *Proceedings of the VLDB Endowment*, 2018.
- [42] P. Barnaghi, W. Wang, C. Henson, and K. Taylor. Semantics for the internet of things. *International Journal on Semantic Web and Information Systems*, 2012.
- [43] T. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 1993.
- [44] E. A. Lee. The past, present and future of cyber-physical systems. *IEEE Transactions on Industrial Informatics*, 2015.

- [45] R. Rajkumar, I. Lee, L. Sha, and J. Stankovic. Cyber-physical systems: The next computing revolution. *Design Automation Conference*, 2010.
- [46] N. Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press, 1948.
- [47] W. R. Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956.
- [48] F. Tao, H. Zhang, A. Liu, and A. Nee. Digital twin in industry. *IEEE Transactions on Industrial Informatics*, 2019.
- [49] A. Kusiak. Smart manufacturing. *International Journal of Production Research*, 2018.
- [50] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 2016.
- [51] M. Satyanarayanan. The emergence of edge computing. *Computer*, 2017.
- [52] M. Villari, M. Fazio, S. Dustdar, O. Rana, and R. Ranjan. Osmotic computing. *IEEE Cloud Computing*, 2018.
- [53] P. Barnaghi, A. Sheth, and C. Henson. From data to actionable knowledge in iot. *IEEE Intelligent Systems*, 2012.
- [54] C. Pahl. Containerization and the paas cloud. *IEEE Cloud Computing*, 2017.
- [55] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 2016.
- [56] Q. Zhang and M. Chen. Kubernetes-based cloud systems. *IEEE Transactions on Cloud Computing*, 2018.
- [57] Q. Zhang, M. Chen, L. Li, et al. Cloud computing: State-of-the-art and research challenges. *Journal of Internet Services and Applications*, 2010.
- [58] R. Buyya, C. S. Yeo, and S. Venugopal. Market-oriented cloud computing. *Future Generation Computer Systems*, 2009.
- [59] D. Bernstein. Containers and cloud: From lxc to docker. *IEEE Cloud Computing*, 2014.
- [60] S. Li, L. Xu, and S. Zhao. The internet of things: A survey. *Information Systems Frontiers*, 2020.

- [61] L. D. Xu, W. He, and S. Li. Internet of things in industries. *IEEE Transactions on Industrial Informatics*, 2014.
- [62] B. Fitzgerald and K. Stol. Continuous software engineering. *Journal of Systems and Software*, 2017.
- [63] C. Pahl. Microservices: A systematic mapping study. *Software: Practice and Experience*, 2017.
- [64] J. Bosch. Continuous software engineering. *Journal of Systems and Software*, 2014.
- [65] B. Fitzgerald. Software engineering and the internet. *Communications of the ACM*, 2013.
- [66] M. Armbrust et al. A view of cloud computing. *Communications of the ACM*, 2010.
- [67] B. Fitzgerald. The transformation of open source software. *MIS Quarterly*, 2006.
- [68] E. S. Raymond. *The Cathedral and the Bazaar*. O’Reilly Media, 1999.
- [69] Ludwig von Bertalanffy. *General System Theory: Foundations, Development, Applications*. George Braziller, 1968.
- [70] R. Baheti and H. Gill. Cyber-physical systems. *The Impact of Control Technology*, 2011.
- [71] Pedro H. J. Nardelli. *Cyber-Physical Systems: Theory, Methodology, and Applications*. Wiley, Hoboken, 2022.
- [72] K. Ogata. *Modern Control Engineering*. Prentice Hall, 2010.
- [73] A. S. Tanenbaum and M. van Steen. *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2007.
- [74] P. T. Eugster et al. The many faces of publish/subscribe. *ACM Computing Surveys*, 2003.
- [75] M. Satyanarayanan. The emergence of edge (fog) computing. *Computer*, 2017.
- [76] R. Light. Mosquitto: Mqtt implementation. *Journal of Open Source Software*, 2017.
- [77] S. Vinoski. Advanced message queuing protocol. *IEEE Internet Computing*, 2006.
- [78] N. Narkhede et al. *Kafka: The Definitive Guide*. O’Reilly, 2017.
- [79] M. Stonebraker. The end of an architectural era. *VLDB*, 2018.

- [80] B. Sigelman. Dapper: Distributed tracing. *Google Research*, 2010.
- [81] V. Medel et al. Kubernetes resource management. *IEEE Cloud Computing*, 2016.
- [82] C. Pahl. Containerization and the cloud. *IEEE Cloud Computing*, 2015.
- [83] A. Carzaniga. Event notification service. *ACM Transactions*, 2001.